

Roteiro aula de Modularização e Funções em C

Slide 1: Introdução - Dividir para Conquistar

"Bom noite, pessoal, meu nome é Giovanna Furlan! Hoje vamos aprender uma técnica fundamental para qualquer programador profissional: a Modularização. O segredo para resolver grandes problemas na programação não é tentar fazer tudo de uma vez, mas sim 'dividir para conquistar', transformando um código gigante em pequenos blocos organizados chamados Funções."

Slide 2: O Problema do Código Espaguete

"Vejam a diferença nestas duas telas. À esquerda, temos tudo amontoadado dentro da função ``main()``. Isso é o que chamamos de Código Espaguete: é difícil de ler, você se perde no meio de 500 linhas e, se houver um erro, é um pesadelo achar. À direita, temos um código modularizado. Percebam como a ``main()`` fica limpa, parecendo um índice de um livro, chamando apenas o que é necessário, como ``mostrarMenu()`` ou ``gerarRelatorio()``."

Slide 3: O que é uma Função? (A Máquina de Suco)

"Para entender a lógica, pensem em uma máquina de fazer suco.

1. Entrada: Você coloca as laranjas (que são os nossos parâmetros).
2. A Máquina: Ela faz o trabalho de espremer (o corpo do código).
3. Saída: Ela te entrega o suco pronto (que é o nosso return)."

Slide 4: A Anatomia de uma Função

"Vamos olhar para o código de uma função de soma. Ela tem quatro partes vitais:

- * Tipo de Retorno: O que ela vai devolver? (ex: ``int``).
- * Nome: Como vamos chamá-la? (ex: ``soma``).
- * Parâmetros: Quais dados ela precisa para trabalhar? (ex: ``int a, int b``).
- * Return: O comando que entrega o 'suco' (resultado) de volta para quem pediu."

Slide 5: Funções Simples (``void``)

"Nem toda função precisa devolver algo. Quando queremos que a função apenas execute uma ação, como imprimir um 'Olá' na tela, usamos o tipo ``void``, que significa 'vazio' em inglês. Como ela não calcula nada para nos devolver, ela não tem o comando ``return``."

Slide 6: Passando os Ingredientes (Parâmetros)

"Os parâmetros são como variáveis temporárias que a função cria para receber dados de fora. Um detalhe importante: elas só existem enquanto a função está rodando. Assim que a função termina, essas variáveis desaparecem da memória."

Slide 7: Devolvendo Dados com o ``return``

"Vejam o ciclo completo: nós enviamos os valores (ex: 5 e 3), a função faz a conta e o ``return`` 'joga' o resultado de volta. Atenção: Se a função devolve um valor, você precisa de uma variável na ``main()`` para 'segurar' esse valor, como em ``int resultado = soma(5, 3);``."

Slide 8: Desafio - Complete a Função

"Olhem para este código de multiplicação. Se a função recebe dois ``int`` e o resultado é ``a * b``, o que deve substituir os '???' no início? A resposta correta é ``int``.

* Por que está certo? Porque se a função vai devolver um número inteiro, o tipo de retorno declarado deve ser ``int``, e não ``void`` ou ``float``."

Slide 9: Modularização de Verdade (Arquivos .h e .c)

"Na vida real, separamos o código em arquivos.

* O arquivo ``h`` (header) funciona como o 'cardápio' ou menu, contendo apenas os nomes das funções.

* O arquivo ``c`` é a 'cozinha', onde o código realmente é escrito.

* A ``main.c`` usa o ``#include`` para poder 'pedir' os pratos desse menu."

Slide 10: Vantagens de Modularizar

"Por que fazemos isso?

1. Reutilização (DRY): 'Don't Repeat Yourself'. Escreva a função uma vez e use mil vezes.

2. Trabalho em Equipe: Um programador faz a parte de matemática, outro a de relatórios, e depois juntamos tudo.

3. Fácil de Consertar: Se a soma der erro, você sabe exatamente em qual arquivo procurar."

Slide 11: Desafio - Caça ao Bug!

"Por que este código da função ``dobrar`` daria erro ao compilar?

* O Erro: A função prometeu devolver um ``int``, mas o corpo da função não tem a linha ``return result;``.

* Consequência: O computador vai devolver um 'lixo de memória' ou dar erro, porque você prometeu um resultado e não entregou nada."

Slide 12: Resumo (Sua Folha de Cola)

"Para não esquecer: toda função segue o padrão [TIPO] [NOME] ([PARÂMETROS]) {
Lógica... ``return [VALOR];`` }*

. Se não devolve nada, é ``void``. Se devolve, o tipo tem que bater com o valor do ``return``."

Slide 13: Pronto para Modularizar!

"Agora que vocês entenderam que funções são como pequenas máquinas organizadas, é hora de praticar! Vamos transformar nossos códigos gigantes em módulos inteligentes."

Muito obrigada pela atenção!